

Rekonstrukcja obrazów tomografii komputerowej za pomocą sieci neuronowej głębokiej na podstawie syntetycznego zbioru uczącego

Autor: **Kamil Cisek**

Promotor: prof. dr hab. inż. Waldemar Smolik, Instytut
Radioelektroniki i Technik Multimedialnych

Seminarium Dyplomowe Inżynierskie

8 grudnia 2025

Plan prezentacji

- 1 Tomografia komputerowa - wprowadzanie do tematyki pracy
- 2 Metody rekonstrukcji obrazów
- 3 Zbiór uczący
- 4 Architektura sieci neuronowej
- 5 Trening sieci neuronowej
- 6 Ewaluacja modelu

Tomografia komputerowa (rentgenowska)

- **Tomografia** - technika obrazowania przekrojów obiektów na podstawie pomiarów dokonywanych z zewnątrz.
- Pomiar polega na rejestracji osłabienia promieniowania X przez obiekt.
- Na podstawie danych (tzw. projekcji) rekonstruuje się obraz przekroju numerycznie.



Rysunek: Tomograf komputerowy [1]

Problem prosty

Jeśli znamy transformację R , to dla przyjętego obiektu f możemy wyznaczyć jego projekcje:

$$g = R(f)$$

gdzie $f(x, y)$ - obiekt, $g(r, \theta)$ - projekcje.

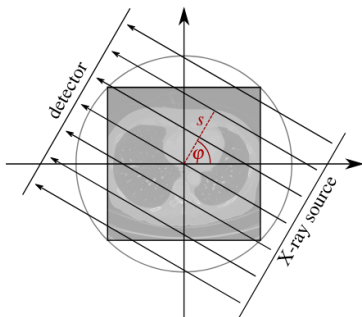
Problem odwrotny

Znając projekcje g , szukamy obrazu f :

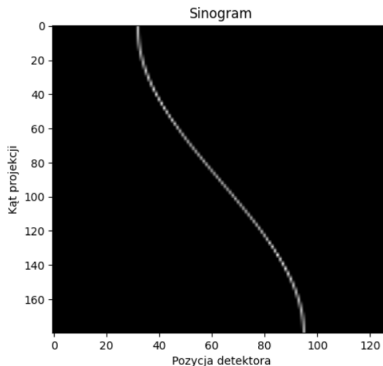
$$f = R^{-1}(g)$$

Projekcje tomograficzne

- Projekcje tworzą tzw. **sinogram**.



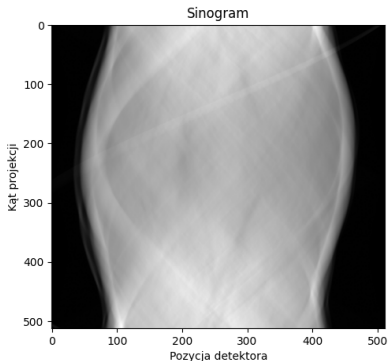
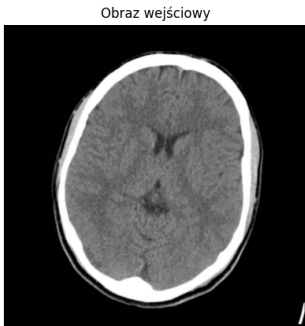
Rysunek: Wiązka równoległa [2]



Rysunek: Sinogram punktu

Problem prosty i odwrotny

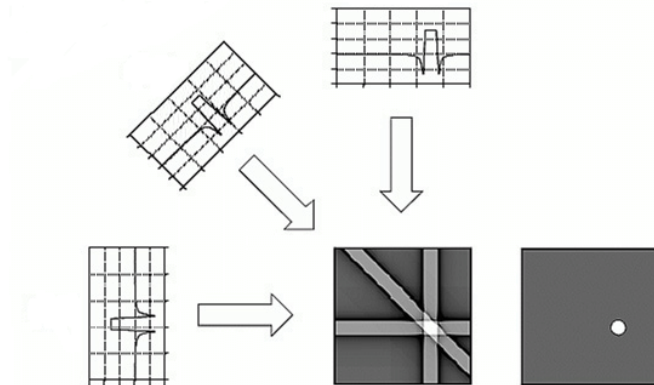
- Aby zrekonstruować przekrój obiektu na podstawie pomiarów tomograficznych należy dokonać transformacji odwrotnej.
- Uwzględniając wszystkie efekty fizyczne (utwardzanie wiązki) mamy do czynienia z transformacją nieliniową.



Rysunek: Obiekt (f) i projekcje (g)

Filtrowana projekcja wsteczna (FBP)

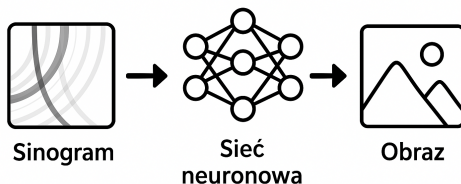
- Filtrowana - przed projekcją wsteczną aplikujemy na sinogramie filtr, aby wzmocnić wysokie częstotliwości.
- Projekcja wsteczna:



Rysunek: Algorytm Projektacji Wstecznej [3]

Cel zastosowania uczenia maszynowego

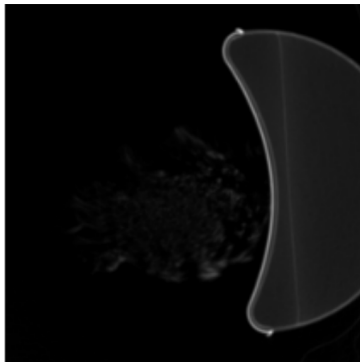
- Szybka rekonstrukcja po wytrenowaniu, mimo czasochłonnego procesu uczenia.
- Redukcja artefaktów i szumów w porównaniu do klasycznych metod.
- Lepsza jakość rekonstrukcji przy niskich dawkach promieniowania.
- Modelowanie transformacji nieliniowej.



Rysunek: Rekonstrukcja za pomocą uczenia maszynowego

Generacja zbioru uczącego

- Zbiór składa się z par sinogram - obraz przekroju.
- Obrazy tomograficzne - uzyskane od promotora, 506 tysięcy obrazów o rozmiarze 512×512 pikseli. Format DICOM.
- Po odrzuceniu obrazów nienaturalnie obciętych, niskiej jakości, z widocznymi artefaktami, z nieprawidłowymi wartościami pikseli - zostało 258 tysięcy.

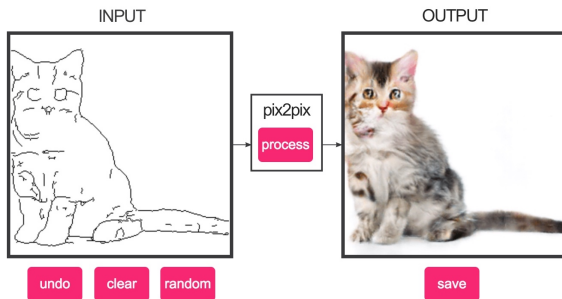


Rysunek: Przykład odrzuconego obrazu

- Sinogramy zostały wygenerowane z użyciem **ASTRA Toolbox** w języku **Python**.
- Obrazy tomograficzne przeskalowane do 256×256 pikseli (65536 pikseli).
- Sinogramy złożone z projekcji pod 512 kątami, każda projekcja składa się z 365 pomiarów (186880 pikseli).
- Geometria wiązki równoległej.
- Zapis zbioru danych w formacie **HDF5** (Hierarchical Data Format), rozmiar 293 GB.

Model Pix2Pix

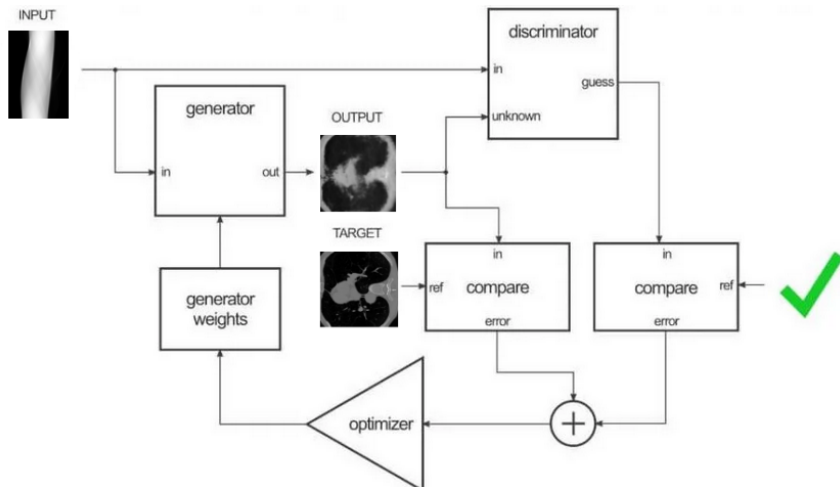
- Generative Adversarial Networks (GAN) - sieci generatywne z adwersarzem.
- Składają się z generatora i dyskryminatora.
- Model warunkowanej sieci GAN do przetwarzania obrazów - Pix2Pix [4].



Rysunek: Obraz wejściowy i wygenerowany przez Pix2Pix [5]

[4] Isola, Phillip et. al. (2017). Image-to-Image Translation with Conditional Adversarial Networks.

Ogólna architektura Pix2Pix



Rysunek: Ogólna architektura modelu Pix2Pix, na podstawie [6]

- **Generator** minimalizuje sumę dwóch składników:

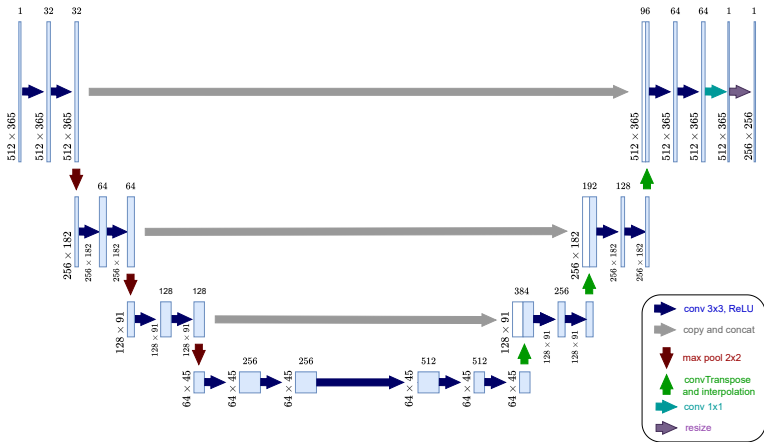
$$\mathcal{L}_G = \text{BCE}(D(G(z)), 1) + \alpha \|G(z) - x_{\text{real}}\|_1.$$

Pierwszy człon wymusza, aby dyskryminator klasyfikował próbki wygenerowane jako prawdziwe, drugi dąży do zmniejszenia różnicy pomiędzy próbką wygenerowaną a rzeczywistą.

- **Dyskryminator** minimalizuje średnią z dwóch osobnych strat BCE:

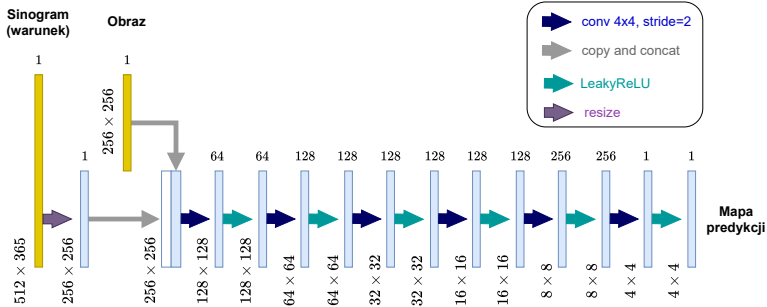
$$\mathcal{L}_D = \frac{1}{2} [\text{BCE}(D(G(z)), 0) + \text{BCE}(D(x_{\text{real}}), 1)].$$

Architektura generatora Pix2Pix



Rysunek: Architektura generatora (G) Pix2Pix - sieć U-Net o 7,34 milionach parametrów

Architektura dyskryminatora Pix2Pix



Rysunek: Architektura dyskryminatora (D) Pix2Pix - sieć konwolucyjna o 1,19 milionach parametrów

- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

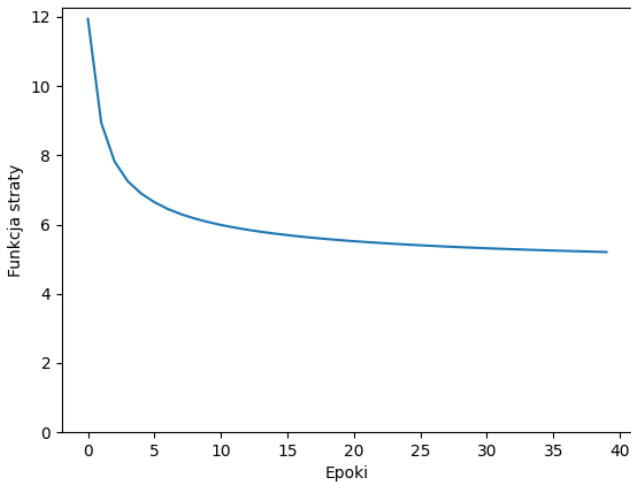
- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

- Implementacja sieci neuronowej w **Pythonie** używając biblioteki **PyTorch**.
- Podział zbioru na treningowy, testowy i walidacyjny - 80/10/10.
- Optymalizator **Adam**.
- Uczenie na karcie graficznej NVIDIA RTX 3090 z 24 GB pamięci VRAM.
- Rozmiar batcha = 16.
- Liczba epok treningowych = 40.
- Czas treningu jednej epoki - około 2 godziny.

Implementacja pętli uczenia modelu

```
1 for epoch in range(epochs):
2     generator.train()
3     discriminator.train()
4
5     for sino, img in train_loader:
6         sino = sino.unsqueeze(1).to(device) # [B, 1, H, W]
7         img = img.unsqueeze(1).to(device)   # [B, 1, H, W]
8
9         fake = generator(sino)
10        fake_pred = discriminator(fake, sino)
11        g_loss = g_criterion(fake, img, fake_pred)
12
13        fake = generator(sino).detach()
14        fake_pred = discriminator(fake, sino)
15        real_pred = discriminator(img, sino)
16        d_loss = d_criterion(fake_pred, real_pred)
17
18        g_optimizer.zero_grad()
19        g_loss.backward()
20        g_optimizer.step()
21
22        d_optimizer.zero_grad()
23        d_loss.backward()
24        d_optimizer.step()
```

Funkcja celu podczas uczenia



Rysunek: Funkcja celu na zbiorze treningowym

- **MSE (Mean Squared Error):**

$$\text{MSE}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

- x_i – wartość piksela obrazu referencyjnego,
- $\hat{x}_i$ – wartość piksela obrazu rekonstruowanego,
- N – liczba pikseli w obrazie.

- **MSE (Mean Squared Error):**

$$\text{MSE}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

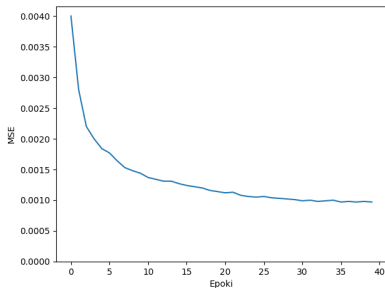
- x_i – wartość piksela obrazu referencyjnego,
- $\hat{x}_i$ – wartość piksela obrazu rekonstruowanego,
- N – liczba pikseli w obrazie.

- **SSIM (Structural Similarity Index):**

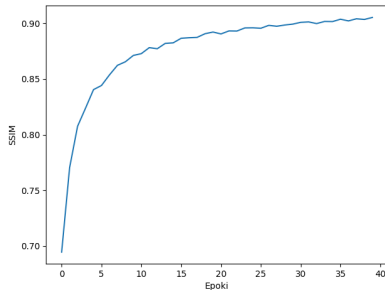
$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

- $\mu_x, \mu_{\hat{x}}$ – średnia jasność obrazów,
- $\sigma_x^2, \sigma_{\hat{x}}^2$ – wariancje,
- $\sigma_{x\hat{x}}$ – kowariancja,
- C_1, C_2 – stałe stabilizujące.

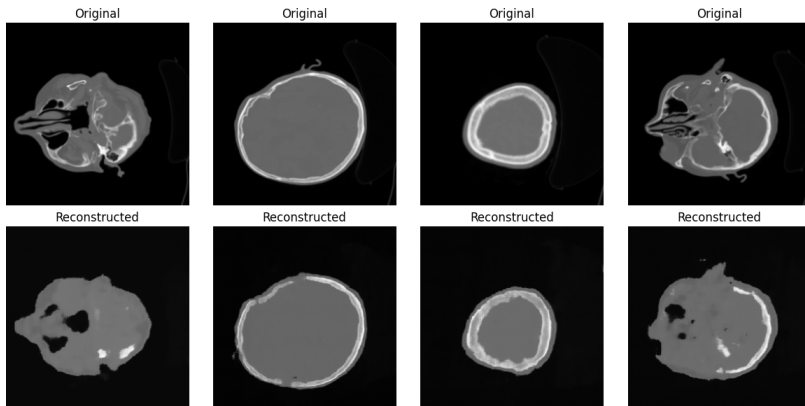
Metryki na zbiorze walidacyjnym w czasie uczenia



Rysunek: Wartość błędu średniokwadratowego na zbiorze walidacyjnym



Rysunek: Wartość wskaźnika SSIM na zbiorze walidacyjnym



Rysunek: Wyniki rekonstrukcji na zbiorze testowym dla przykładowych obrazów

- Zmiana funkcji celu - dodanie komponentu uwzględniającego wskaźnik MSE.
- Przejście na połówkową precyzję (FP16) i powiększenie modelu.
- Dodanie szumu do sinogramów przy treningu.
- Generacja syntetycznego zbioru uczącego na podstawie obrazów naturalnych, trening modelu i porównanie rezultatów na obrazach tomograficznych z modelem wytrenowanym wcześniej.
- Dokładniejsza ewaluacja modelu.

- Zmiana funkcji celu - dodanie komponentu uwzględniającego wskaźnik MSE.
- Przejście na połówkową precyzję (FP16) i powiększenie modelu.
- Dodanie szumu do sinogramów przy treningu.
- Generacja syntetycznego zbioru uczącego na podstawie obrazów naturalnych, trening modelu i porównanie rezultatów na obrazach tomograficznych z modelem wytrenowanym wcześniej.
- Dokładniejsza ewaluacja modelu.

- Zmiana funkcji celu - dodanie komponentu uwzględniającego wskaźnik MSE.
- Przejście na połówkową precyzję (FP16) i powiększenie modelu.
- Dodanie szumu do sinogramów przy treningu.
- Generacja syntetycznego zbioru uczącego na podstawie obrazów naturalnych, trening modelu i porównanie rezultatów na obrazach tomograficznych z modelem wytrenowanym wcześniej.
- Dokładniejsza ewaluacja modelu.

- Zmiana funkcji celu - dodanie komponentu uwzględniającego wskaźnik MSE.
- Przejście na połówkową precyzję (FP16) i powiększenie modelu.
- Dodanie szumu do sinogramów przy treningu.
- Generacja syntetycznego zbioru uczącego na podstawie obrazów naturalnych, trening modelu i porównanie rezultatów na obrazach tomograficznych z modelem wytrenowanym wcześniej.
- Dokładniejsza ewaluacja modelu.

- Zmiana funkcji celu - dodanie komponentu uwzględniającego wskaźnik MSE.
- Przejście na połówkową precyzję (FP16) i powiększenie modelu.
- Dodanie szumu do sinogramów przy treningu.
- Generacja syntetycznego zbioru uczącego na podstawie obrazów naturalnych, trening modelu i porównanie rezultatów na obrazach tomograficznych z modelem wytrenowanym wcześniej.
- Dokładniejsza ewaluacja modelu.

Dziękuję za uwagę!

- 1 <https://lukasz.med.pl/szerokodetektorowy-tomograf-komputerowy-juz-dostepny-dla-pacjentow/>
- 2 Leuschner J, Schmidt M, Baguer DO, Maass P. LoDoPaB-CT, a benchmark dataset for low-dose computed tomography reconstruction. (2021).
- 3 Qiu, Dejian and Seeram, Euclid. Does Iterative Reconstruction Improve Image Quality and Reduce Dose in Computed Tomography? (2016).
- 4 Isola, Phillip et. al. (2017). Image-to-Image Translation with Conditional Adversarial Networks.
- 5 <https://phillipi.github.io/pix2pix/>
- 6 <https://neurohive.io/en/popular-networks/pix2pix-image-to-image-translation/>